

MICROCONTROLLERS

Labo 5

Naslag

Gegenereerd op 31/07/2026 uit tdmts.github.io/Microcontrollers

Inhoud

Soorten motoren

[De servomotor](#)

[De DC motor](#)

[De stappenmotor](#)

Een motor aansturen

[De transistor als schakelaar](#)

[De H-brug](#)

Datasheets

[P2N2222A \(2N2222\) los document](#)

[L293D los document](#)

[NASLAG](#)

De servomotor

Een standaard servomotor is een **positiemotor**. Je zegt hem welke hoek je wil, en hij draait daarheen en houdt die stand vast. Het bereik is ongeveer 180 graden.

Dat is meteen het verschil met de andere motoren in dit labo. Bij een DC motor stuur je *hoe hard* hij draait, bij een stappenmotor stuur je *hoeveel stappen* hij zet. Bij een servo stuur je een **hoek**, en de elektronica in de servo zorgt er zelf voor dat de as daar terechtkomt en blijft.

Hoe je een servo aanstuurt

Een servo heeft drie draden: massa (meestal bruin of zwart), voeding (rood) en een **stuurdraad** (meestal oranje of geel). Over die stuurdraad stuur je pulsen, en de **breedte van de puls** bepaalt de hoek:

Pulsbreedte	Hoek
1 ms	0 graden
1,5 ms	90 graden
2 ms	180 graden

Die puls moet ongeveer **50 keer per seconde** herhaald worden. Zolang de pulsen blijven komen, houdt de servo zijn positie vast en verzet hij zich wanneer je tegen de as duwt. Stoppen de pulsen, dan laat hij los en kan je de as met de hand ronddraaien.

⚠ DIT IS NIET HETZELFDE ALS ANALOGWRITE()

Je hebt in `analogWrite()` ook met pulsen gewerkt, maar dat signaal is hier onbruikbaar. `analogWrite()` stuurt ongeveer **490 keer per seconde** een puls, en je stelt er de *verhouding* aan/uit mee in, niet de breedte in milliseconden. Een servo wil precies 50 pulsen per seconde van 1 tot 2 ms. Dat zijn twee verschillende signalen die allebei toevallig "PWM" heten.

i WAAR DIE 1 TOT 2 MILLISECONDEN VANDAAN KOMEN

Die maat komt uit de modelbouw. Een radiobestuurde vliegtuig heeft één zender en één ontvanger, maar meerdere servo's: een voor het richtingsroer, een voor de hoogteroeren, een voor het gas. De zender stuurt daarom om de 20 milliseconden een reeks pulsen door, één per kanaal na elkaar. De ontvanger knipt die reeks uiteen en geeft elke puls aan de servo waar ze bij hoort. De breedte van zo'n puls, tussen 1 en 2 ms, gaf de stand van de stuurknuppel weer.

Wat jij hier op je stuurdraad zet is dus één kanaal uit die radioreeks, en de 50 pulsen per seconde zijn de 20 milliseconden waarin al die kanalen samen pasten. Die afspraak dateert van de jaren 60 en is sindsdien niet meer aangepast, zodat een servo van gelijk welk merk en gelijk welk bouwjaar op hetzelfde signaal draait. Ook de Servo-bibliotheek werkt er nog mee: `write(0)` en `write(180)` worden intern 544 en 2400 microseconden, de uiterste pulsbreedtes die ze wil versturen.

De Servo-bibliotheek

Die pulsen zelf timen met `digitalWrite()` en `delayMicroseconds()` kan, maar dan is je programma de hele tijd bezig met tellen. Daarom gebruik je de **Servo**-bibliotheek. Hoe je een bibliotheek toevoegt lees je op [Bibliotheken gebruiken](#).

De bibliotheek zet op de achtergrond een timer aan het werk, die de pulsen genereert terwijl jouw `loop()` gewoon doorloopt. Je hoeft dus nooit zelf een puls te tellen.

i EEN SERVO HOEFT NIET OP EEN PIN MET EEN ~

De Servo-bibliotheek maakt de pulsen zelf en werkt op **elke digitale pin** van de UNO. De ~-pinnen zijn de pinnen waar `analogWrite()` op werkt, en dat is hier een ander signaal.

Het omgekeerde geldt wel. Zodra je op een UNO één servo aankoppelt, gebruikt de bibliotheek de timer van pin 9 en pin 10. Op die twee pinnen werkt `analogWrite()` dan niet meer, ook al staat er een ~ bij. Heb je in dezelfde schakeling nog een led te dimmen, kies daar dan pin 3, 5, 6 of 11 voor.

De functies

Functie	Wat ze doet
<code>attach(pin)</code>	Koppelt het servo-object aan een pin. Doe dit in de <code>setup()</code> .
<code>write(hoek)</code>	Stuurt de servo naar een hoek tussen 0 en 180.
<code>writeMicroseconds(us)</code>	Zelfde, maar je geeft de pulsbreedte rechtstreeks op: 1000 is 0 graden, 1500 is 90, 2000 is 180.
<code>read()</code>	Geeft de laatste hoek terug die je geschreven hebt.
<code>attached()</code>	Geeft terug of het object aan een pin gekoppeld is.
<code>detach()</code>	Laat de pin weer los. De pulsen stoppen en de servo laat zijn positie los.

Een servo naar een vaste hoek sturen

Bovenaan je programma voeg je de bibliotheek toe en maak je een servo-object. In de `setup()` koppel je dat object aan de pin waar de stuurdraad op zit.

```
1  #include <Servo.h>
2
3  Servo mijnServo;
4
5  const int servoPin = 9;
6
7  void setup()
8  {
9      mijnServo.attach(servoPin);
10 }
11
12 void loop()
13 {
14     mijnServo.write(0);
15     delay(1000);
16
17     mijnServo.write(90);
18     delay(1000);
19
20     mijnServo.write(180);
21     delay(1000);
22 }
```

De servo springt hier van stand naar stand. Tussen de `write()` en de volgende moet je hem wel de tijd geven om er te geraken, en dat is waar de `delay()` voor dient. Zonder die wachttijd schrijf je een nieuwe hoek voor de as de vorige bereikt heeft.

Vloeiend heen en weer (sweep)

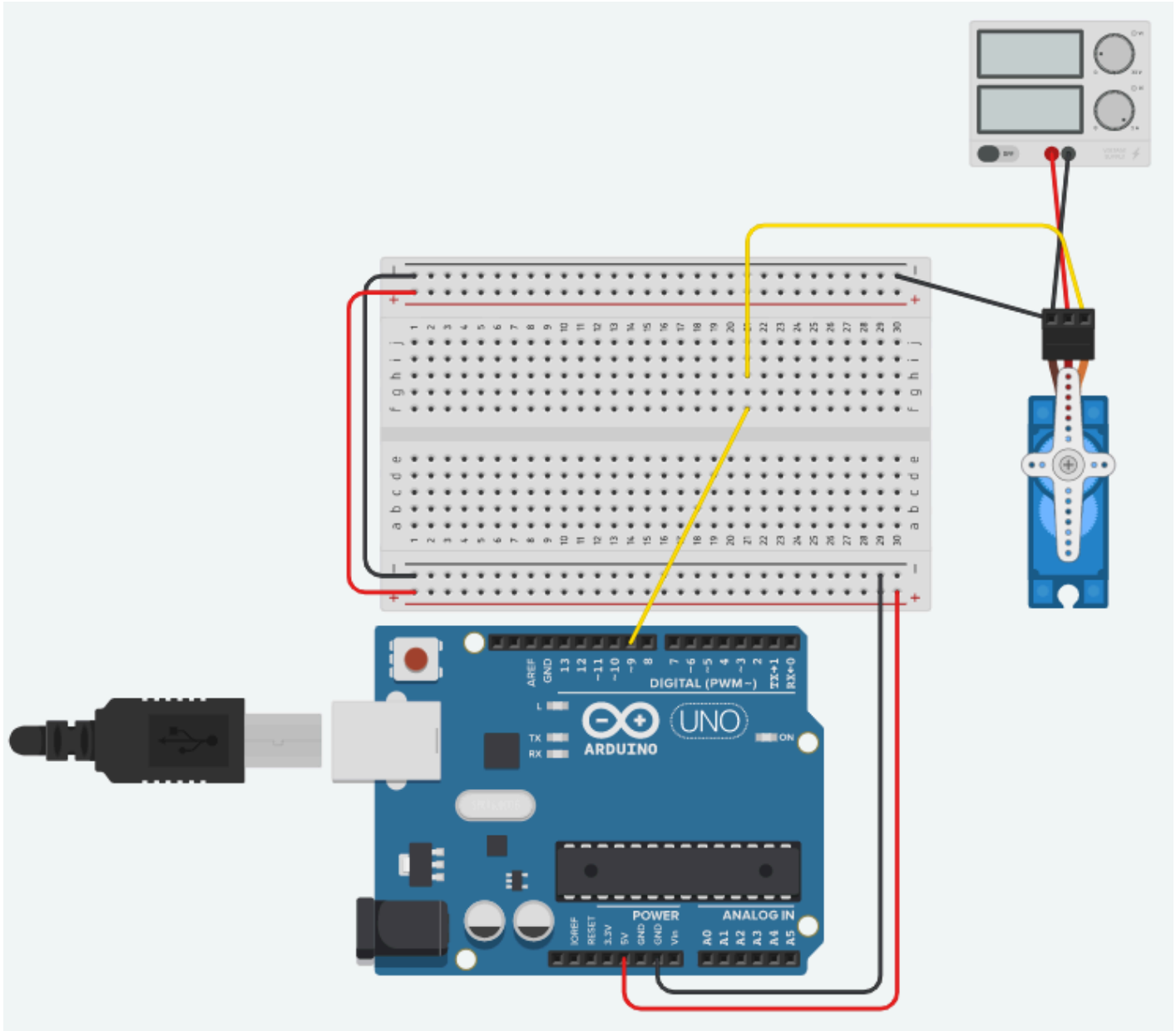
Wil je de as vloeiend zien bewegen in plaats van springen, stuur je hem in stappen van 1 graad met een korte pauze ertussen. Dat is wat een `for`-lus doet.

```
1  #include <Servo.h>
2
3  Servo mijnServo;
4
5  const int servoPin = 9;
6
7  void setup()
8  {
9      mijnServo.attach(servoPin);
10 }
11
12 void loop()
13 {
14     // van 0 naar 180 graden
15     for (int hoek = 0; hoek <= 180; hoek++)
16     {
17         mijnServo.write(hoek);
18         delay(15); // geef de as tijd om die ene graad af te leggen
19     }
20
21     // en terug
22     for (int hoek = 180; hoek >= 0; hoek--)
23     {
24         mijnServo.write(hoek);
25         delay(15);
26     }
27 }
```

Met 15 ms per graad duurt een volledige sweep van 0 naar 180 ongeveer $180 \times 15 \text{ ms} = 2,7$ seconden. Maak je die wachttijd kleiner, dan gaat de as sneller, tot je op een punt komt waar de servo het mechanisch niet meer volgt en de beweging weer schokkerig wordt.

Voeding

Een kleine hobby servo kan je bij het uitproberen meestal rechtstreeks op de 5 V van de Arduino hangen. Zodra de servo iets moet trekken, of zodra er meerdere servo's aan hangen, trekt hij meer stroom dan de spanningsregelaar van de Arduino kan leveren. Je merkt dat doordat de Arduino spontaan herstart op het moment dat de servo begint te bewegen. Dan heb je een aparte voeding nodig.



[Klik op de afbeelding om te vergroten](#)

⚠ TWEE VOEDINGEN DELEN HUN MASSA

Voed je de servo apart, dan moet de **min** van die voeding aan de **GND** van de Arduino. Zonder die gemeenschappelijke massa heeft de servo geen referentie voor je stuursignaal, want de puls die de Arduino stuurt betekent dan niets voor de servo. Het gevolg is een servo die niet reageert of wild begint te schokken, terwijl je code in orde is.

De DC motor

Een DC motor of gelijkstroommotor is de eenvoudigste elektromotor die er is. Zet er spanning op en de as draait. Hoe hoger de spanning, hoe sneller. Draai je de polariteit om, dan draait hij de andere kant op.

Meer stuurmogelijkheden zijn er niet, en dat maakt hem gemakkelijk om te gebruiken. Een DC motor telt geen omwentelingen en kent geen hoek, dus hij weet zelf niet waar hij staat. Wil je weten hoe ver hij gedraaid heeft, dan moet je dat er zelf bij meten, bijvoorbeeld met een eindeloopschakelaar.

Wat je wil	Wat je doet
Sneller of trager	Hogere of lagere gemiddelde spanning, in de praktijk met PWM
Andere richting	De twee draden van plaats wisselen, in de praktijk met een H-brug
Stoppen	De spanning wegnemen

Waarom je hem niet aan een pin hangt

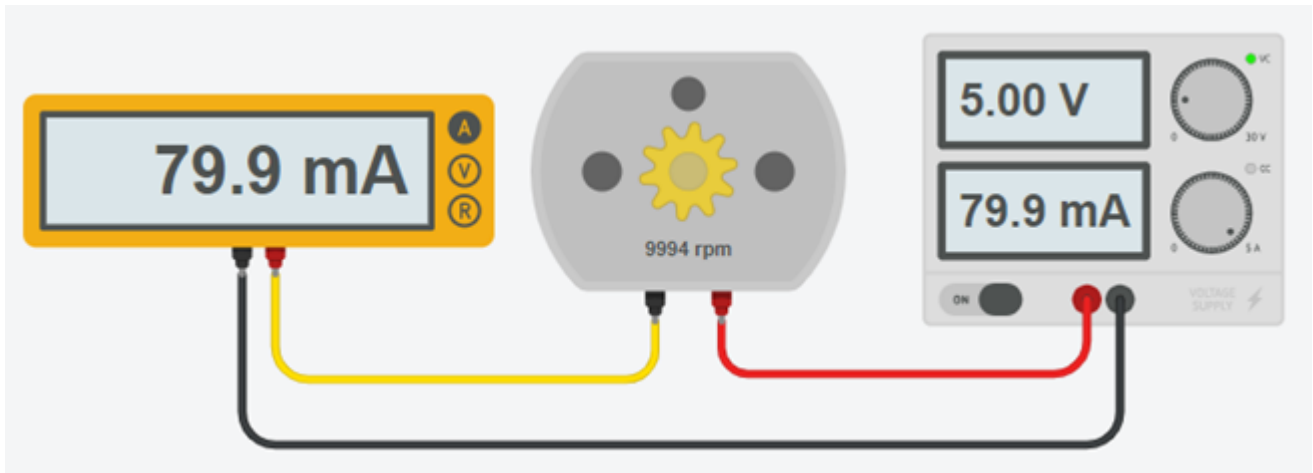
Een led hing je met een weerstand aan een uitgangspin. Bij een motor lukt dat niet. Een digitale pin van de UNO mag maximaal ongeveer **40 mA** leveren, en zelfs dat is een absolute grens waar je liever ver onder blijft. Een kleine hobbymotor trekt al snel **enkele honderden mA**, en op het moment dat hij vanuit stilstand vertrekt nog veel meer.

Hang je hem er toch rechtstreeks aan, dan gebeurt er in het beste geval niets zinnigs en in het slechtste geval sneuvelt de uitgang van je microcontroller. Je hebt dus altijd iets tussen de pin en de motor nodig, dat de stroom uit een andere bron haalt en door de pin alleen laat *schakelen*:

- Een [transistor](#), als de motor maar één kant op moet draaien.
- Een [H-brug](#) zoals de L293D of de L298N, als hij twee kanten op moet.

Hoeveel stroom trekt jouw motor?

Meet het, in plaats van te gokken. Hang de motor even rechtstreeks aan een voeding en meet de stroom met een multimeter, of lees ze af op de voeding zelf. In TinkerCAD werkt dat precies hetzelfde.



[Klik op de afbeelding om te vergroten](#)

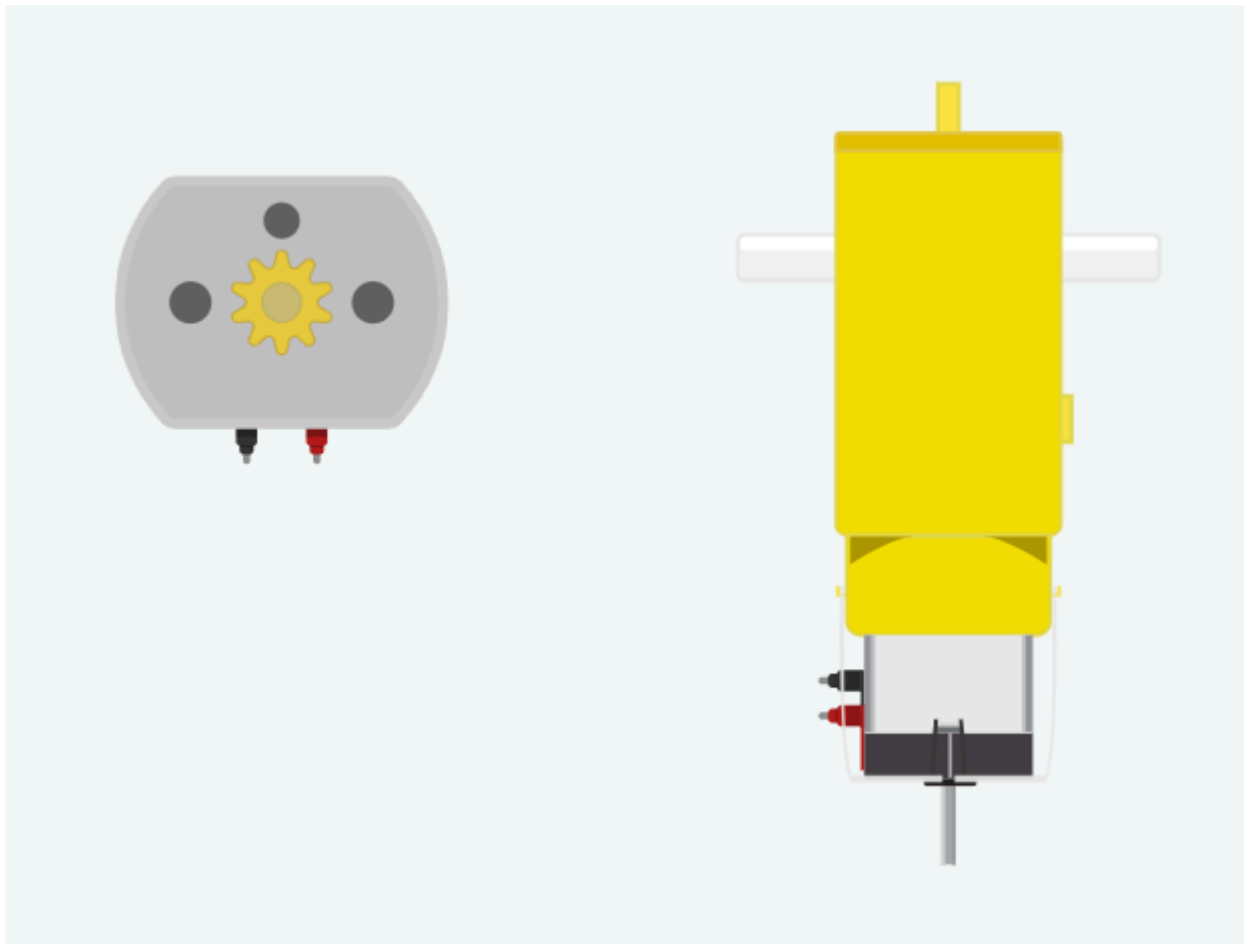
De motor uit dit voorbeeld trekt 80 mA. Dat is dubbel zoveel als een pin ooit mag leveren, en die waarde heb je verderop nodig om je transistor te dimensioneren.

MEET BIJ BELASTING, NIET IN HET LUCHTLEDIGE

Een motor die vrij ronddraait trekt veel minder dan een motor die iets moet trekken. De stroom die je meet met een vrije as is dus een ondergrens. Rekent je schakeling daar krap op, dan werkt ze in TinkerCAD zonder problemen en begeeft ze het zodra er een wiel aan hangt.

In TinkerCAD

Je vindt de DC motor onder **Gelijkstroommotor** en onder **Hobby motorreductor**. Die laatste is een motor met een tandwielkast erop, die trager draait maar meer koppel geeft.



[Klik op de afbeelding om te vergroten](#)

De stappenmotor

Een stappenmotor zet één **stap** per keer, en alleen wanneer je de spoelen in de juiste volgorde bekrachtigt. Doe je niets, dan blijft hij staan waar hij staat.

Dat maakt hem het tegenovergestelde van een [DC motor](#). Een DC motor zet je aan en dan draait hij, maar je weet nooit hoe ver. Een stappenmotor beweegt alleen als jij hem stap voor stap voortduwt, en dus weet je exact hoe ver hij gedraaid heeft, want je hebt de stappen zelf geteld. Daarom zitten ze in printers, plotters en 3D-printers, overal waar een positie moet kloppen zonder dat er een sensor aan te pas komt.

	Wat je stuurt	Weet hij waar hij staat?
Servo	Een hoek	Ja, hij regelt zelf bij
DC motor	Spanning, dus snelheid	Nee
Stappenmotor	Stappen, één voor één	Ja, zolang jij ze telt

De drie motoren van dit labo naast elkaar

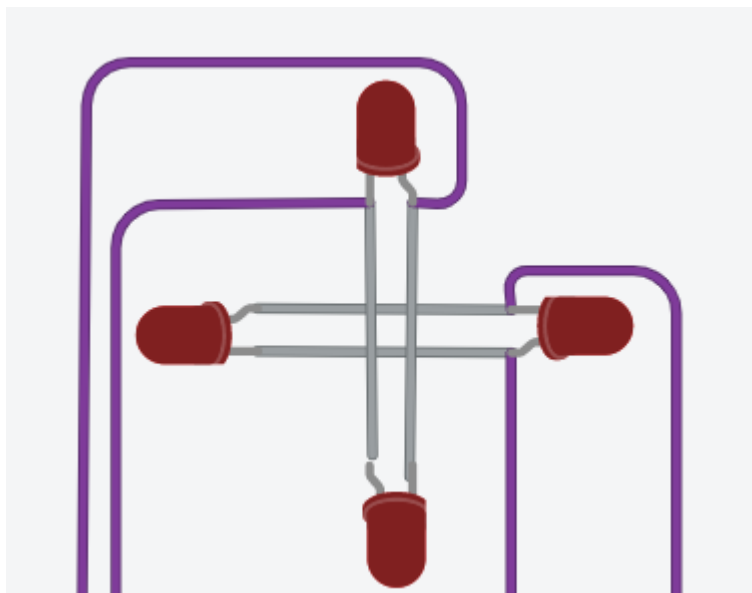
Twee spoelen

Binnenin zitten twee spoelen, die we **A** en **B** noemen, en daartussen een magnetische rotor. Bekrachtig je spoel A, dan draait de rotor naar spoel A toe en blijft daar hangen. Bekrachtig je vervolgens spoel B, dan draait hij een kwartslag verder naar B. Zo trek je hem stap voor stap rond.

Elke spoel kan je in **twee richtingen** bekrachtigen, want de stroom kan er twee kanten door. Dat geeft vier standen om naartoe te trekken: A in de ene richting, B in de ene richting, A in de andere, B in de andere. En omdat je per spoel de stroomrichting moet kunnen kiezen, heb je per spoel een [H-brug](#) nodig. Twee spoelen vragen dus twee bruggen, en die zitten allebei in één L293D.

In TinkerCAD: vier leds in plaats van een motor

TinkerCAD heeft geen bruikbaar model van een stappenmotor. We vervangen de motor daarom door **vier leds**, twee per twee **anti-parallel** geschakeld, waarbij elk paar op de plaats van één spoel staat.



[Klik op de afbeelding om te vergroten](#)

Anti-parallel wil zeggen dat ze naast elkaar staan, maar omgekeerd. Loopt de stroom de ene kant op door het paar, dan brandt de ene led. Loopt hij de andere kant op, dan brandt de andere. Zo zie je aan welke led brandt niet alleen *of* een spoel bekrachtigd is, maar ook **in welke richting**. Een correct draaiende motor herken je aan leds die in de rondte oplichten.

i WAAROM NIET VIER LOSSE LEDS?

Omdat een echte stappenmotor ook maar twee spoelen heeft, en dus maar vier draden. Met vier losse leds zou je vier onafhankelijke uitgangen simuleren die in de echte motor niet bestaan, en dan schrijf je code die op een echte motor niet werkt.

Aansluiten op de L293D

Beide spoelen hangen elk over één brug van de [L293D](#):

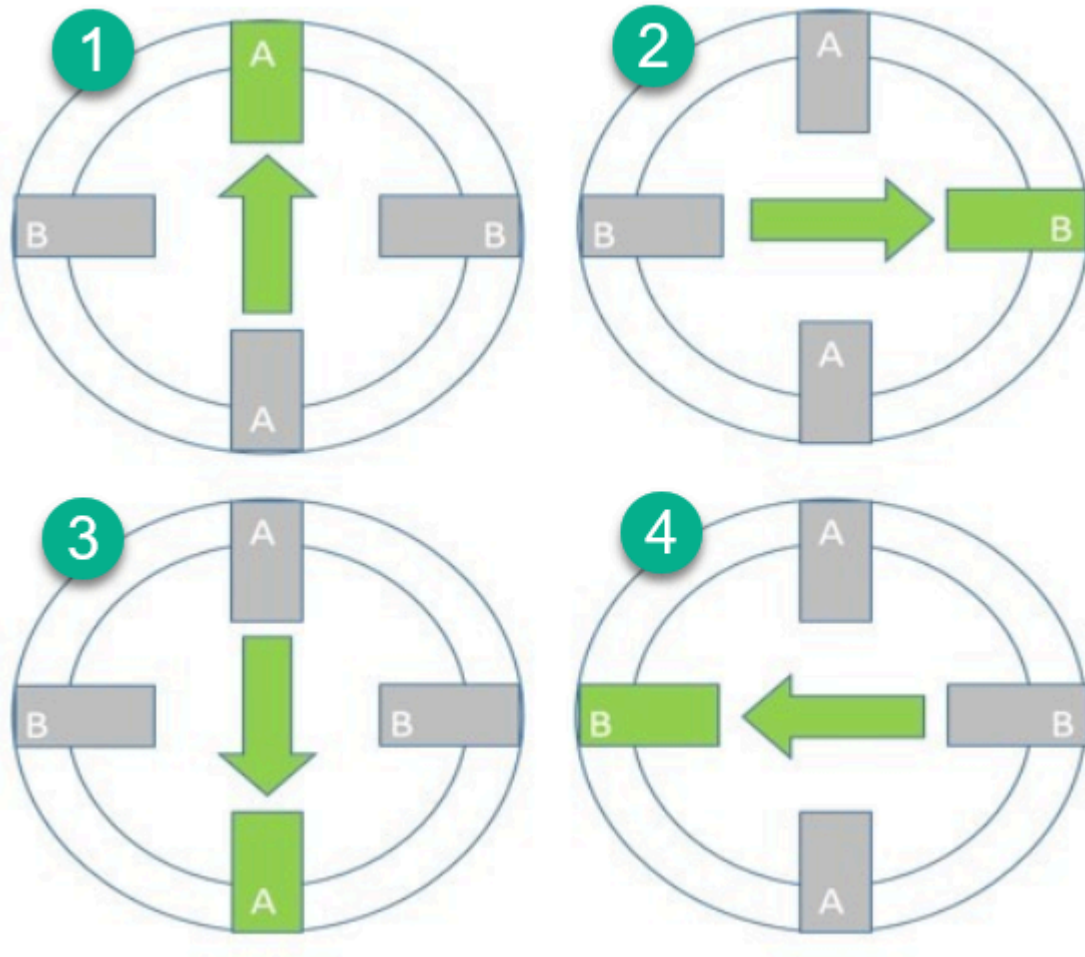
Arduino	L293D-ingang	L293D-uitgang	Waar het naartoe gaat
pin 2	1A (pin 2)	1Y (pin 3)	de ene kant en de andere kant van spoel A
pin 3	2A (pin 7)	2Y (pin 6)	
pin 4	3A (pin 10)	3Y (pin 11)	de ene kant en de andere kant van spoel B
pin 5	4A (pin 15)	4Y (pin 14)	

De twee enable-pinnen (1,2EN en 3,4EN) hang je vast aan 5 V, want bij een stappenmotor wil je de bruggen altijd actief hebben. Je regelt de snelheid hier niet met PWM maar met de tijd tussen twee

stappen.

Full step

De eenvoudigste manier om te sturen heet **one phase, full step**. Je bekrachtigt telkens één spoel, in vier standen na elkaar.



[Klik op de afbeelding om te vergroten](#)

Vertaald naar de vier Arduino-pinnen wordt dat:

Stap	pin 2	pin 3	pin 4	pin 5	Wat er gebeurt
1	HIGH	LOW	LOW	LOW	Spoel A, ene richting
2	LOW	LOW	HIGH	LOW	Spoel B, ene richting
3	LOW	HIGH	LOW	LOW	Spoel A, andere richting
4	LOW	LOW	LOW	HIGH	Spoel B, andere richting

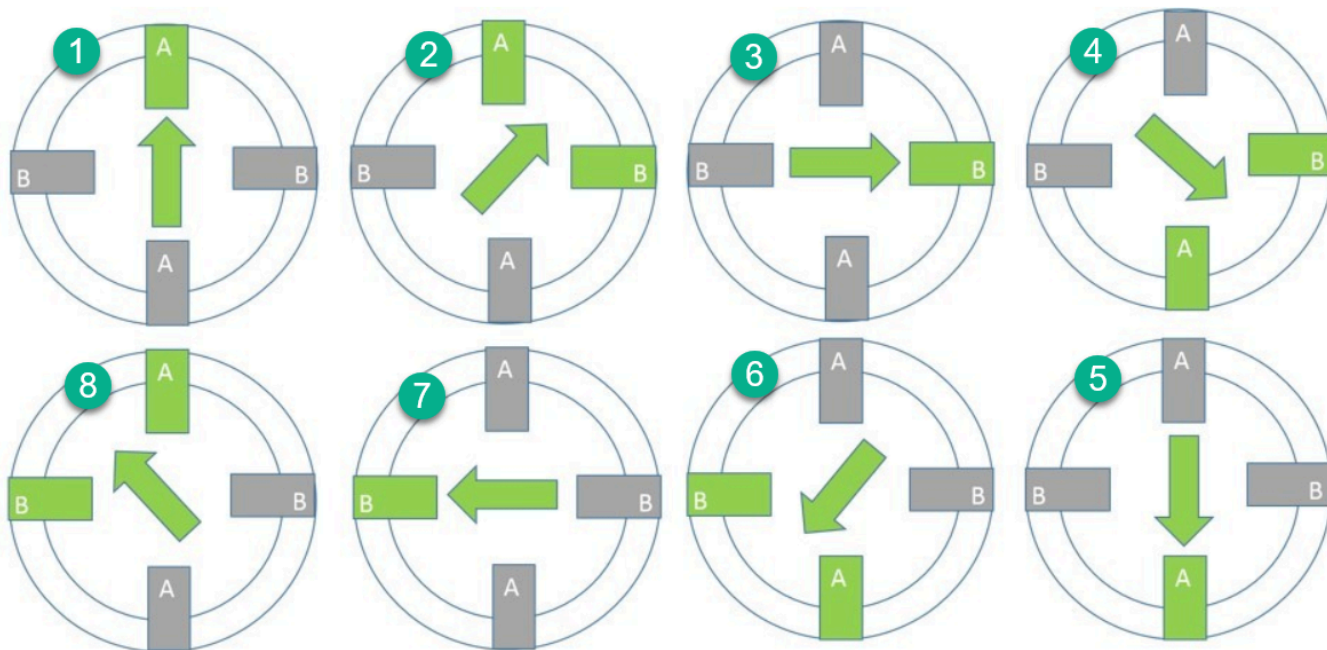
De vier stappen van full step

DE VOLGORDE IS A, B, A, B EN NIET A, A, B, B

Kijk goed naar de tabel. Je wisselt telkens van spoel. Zou je eerst spoel A in beide richtingen doen en dan pas spoel B, dan springt de rotor van 0 naar 180 graden en weer terug naar 90. Hij draait dan niet rond maar trilt heen en weer. Dat is de klassieke fout, en aan je leds zie je het meteen. Die moeten **in de rondte** oplichten en niet om de beurt van links naar rechts.

Half step

Bij full step trek je de rotor telkens een hele stap verder. Je kan hem ook **halverwege** laten stoppen door twee spoelen tegelijk te bekrachtigen, want dan gaat de rotor tussen die twee in staan. Wissel je die twee soorten standen af, dan krijg je **acht** standen in plaats van vier, en dus twee keer zoveel stappen per omwenteling.



[Klik op de afbeelding om te vergroten](#)

Stap	pin 2	pin 3	pin 4	pin 5	Wat er gebeurt
1	HIGH	LOW	LOW	LOW	Alleen spoel A
2	HIGH	LOW	HIGH	LOW	A en B samen
3	LOW	LOW	HIGH	LOW	Alleen spoel B
4	LOW	HIGH	HIGH	LOW	A en B samen
5	LOW	HIGH	LOW	LOW	Alleen spoel A
6	LOW	HIGH	LOW	HIGH	A en B samen
7	LOW	LOW	LOW	HIGH	Alleen spoel B
8	HIGH	LOW	LOW	HIGH	A en B samen

De acht stappen van half step

💡 DE FULL-STEPTABEL ZIT ERIN VERSTOPT

Vergelijk stap 1, 3, 5 en 7 van half step met de vier stappen van full step. Het zijn dezelfde. Half step is dus full step met tussen elke twee stappen een extra stand waarin beide spoelen aan staan.

De stappentabel in code

Die tabellen kan je letterlijk overnemen als `array`. Elke rij is één stap, en per rij staat er voor elke motorpin een 1 of een 0. Dat leest veel beter dan twintig `digitalWrite()`-regels onder elkaar.

```
1  const int motorPinnen[] = {2, 3, 4, 5};
2
3  // Elke rij is één stap. Per rij staat er voor elke motorpin een 1 of een 0.
4  const int volleStap[4][4] = {
5      {1, 0, 0, 0}, // spoel A, ene richting
6      {0, 0, 1, 0}, // spoel B, ene richting
7      {0, 1, 0, 0}, // spoel A, andere richting
8      {0, 0, 0, 1}  // spoel B, andere richting
9  };
```

CPP

Eén stap zetten wordt dan een lus over de vier pinnen:

```
1  void zetStap(int stap)
2  {
3      for (int pin = 0; pin < 4; pin++)
4      {
5          digitalWrite(motorPinnen[pin], volleStap[stap][pin]);
6      }
7  }
```

CPP

En daarmee heb je meteen drie dingen in handen:

- **Draaien** is de stap ophogen en weer bij 0 beginnen na de laatste rij. De **restoperator** `%` doet dat in één regel.
- **Van richting veranderen** is de tabel de andere kant op doorlopen, dus aftellen in plaats van optellen.
- **Sneller of trager** is de wachttijd tussen twee stappen aanpassen.

ER BESTAAT OOK TWO PHASE, FULL STEP

Wij sturen telkens één spoel tegelijk aan, en dat heet voluit *one phase, full step*. Je kan ook altijd twee spoelen samen bekrachtigen. De motor heeft dan evenveel stappen, maar meer koppel, en hij trekt ook dubbel zoveel stroom. In dit labo blijven we bij one phase.

De transistor als schakelaar

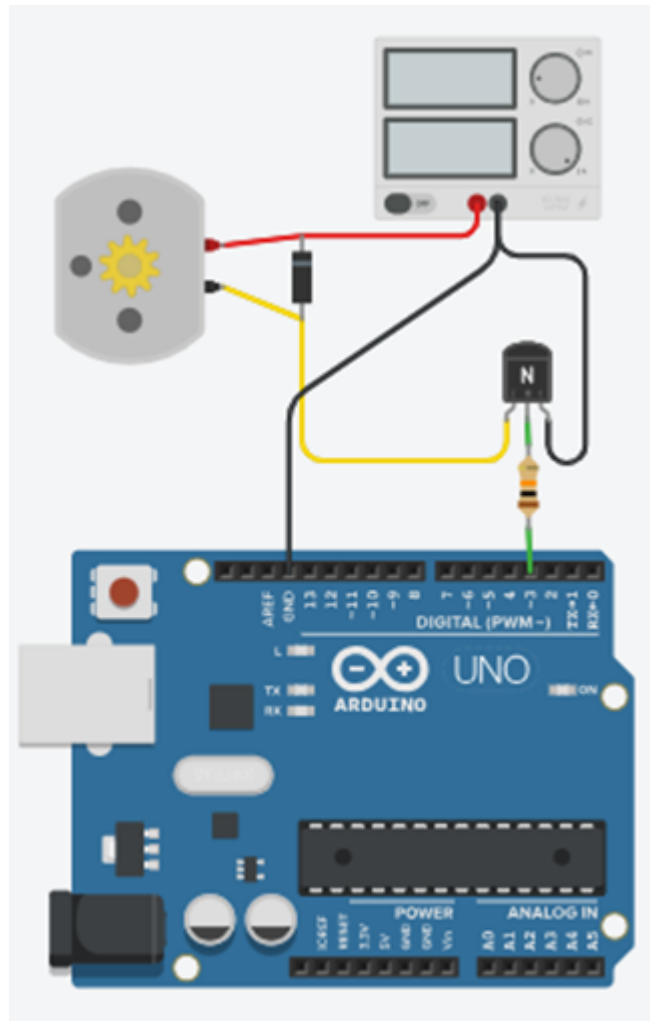
Een [DC motor](#) trekt meer stroom dan een pin van je Arduino mag leveren. Dat probleem en zijn oplossing ken je al uit [Vermogen schakelen](#) in labo 1, waar je drie leds met een transistor schakelde. Hier is de stroom een stuk groter, en komt er één component bij.

De drie aansluitingen

Een NPN-transistor heeft drie aansluitingen. Stuur je een kleine stroom in de **basis**, dan laat hij een veel grotere stroom door tussen **collector** en **emitter**. Bij een motor gaan ze hierheen:

Aansluiting	Waar ze naartoe gaat
B (basis)	Via een weerstand naar de uitgangspin van de Arduino
C (collector)	Naar de motor
E (emitter)	Naar GND

De motor hangt dus **boven** de transistor: van de plus van de voeding naar de collector. De transistor zit tussen de motor en de massa en laat de stroom er al dan niet door. In TinkerCAD is de **2N2222** gemodelleerd, en die gebruiken we hier.



[Klik op de afbeelding om te vergroten](#)

De vrijloopdiode

Kijk op het schema hierboven naar het component dat over de motor heen staat, met de streep naar de plus. Dat is een **diode**.

Een motor is een spoel. Zolang er stroom door loopt zit er energie in het magnetisch veld, en een spoel laat zijn stroom niet zomaar ophouden. Schakelt de transistor uit, dan probeert de spoel die stroom te blijven duwen. Er is geen weg meer, dus de spanning over de spoel loopt op tot ze er wel een vindt, dwars door je transistor heen. Die spanningspiek is een veelvoud van je voedingsspanning en maakt een transistor stuk in een enkele schakeling.

De diode geeft die stroom een uitweg. In normale werking staat ze in sperrichting en doet ze niets. Op het moment dat je uitschakelt, staat ze in geleiding en laat ze de spoelstroom rondlopen tot de energie op is. Vandaar de naam **vrijloopdiode**.

⚠️ ALTIJD EEN DIODE OVER EEN MOTOR OF EEN RELAIS

Dit geldt bij elke spoel die je schakelt: een motor, een relais, een elektromagneet. De diode staat **omgekeerd** over de spoel, dus met de streep aan de kant van de plus. Zet je ze per ongeluk andersom, dan sluit je je voeding kort.

In TinkerCAD kan je de diode weglaten en werkt alles door. Op een echt breadboard is dat het verschil tussen een schakeling die blijft werken en een transistor die na een minuut warm en dood is.

De basisweerstand berekenen

De weerstand tussen je pin en de basis reken je uit zoals in labo 1, met dezelfde twee formules en dezelfde marge op de hFE. Waar die formules vandaan komen en waarom je door 5 deelt, staat op [Vermogen schakelen](#).

$$I_b = \frac{I_c}{h_{FE}} \quad R_b = \frac{V_{cc} - V_{be}}{I_b}$$

Alleen de collectorstroom is anders. Voor de motor van 80 mA uit [De DC motor](#), met $V_{cc} = 5 \text{ V}$, $V_{be} = 0,7 \text{ V}$ en een hFE van 200 die we tot 40 herleiden:

$$I_b = \frac{0,08 \text{ A}}{40} = 0,002 \text{ A} = 2 \text{ mA}$$
$$R_b = \frac{5 \text{ V} - 0,7 \text{ V}}{0,002 \text{ A}} = 2150 \text{ } \Omega$$

Die waarde bestaat niet als standaardweerstand, dus je neemt de dichtstbijzijnde die je hebt liggen: 2,2 k Ω . Vbe en hFE zoek je op in de [datasheet van de transistor](#).

Wat de transistor niet kan

Met deze schakeling kan je de motor aan- en uitzetten, en met `analogWrite()` op de basis ook zijn snelheid regelen. Wat je er niet mee kan, is de motor de andere kant op laten draaien, want daarvoor zou je de twee draden van plaats moeten wisselen en één transistor doet dat niet. Daar heb je een [H-brug](#) voor nodig.

De H-brug

Een H-brug is een schakeling die een stroom kan leveren waarvan je de **richting** kiest. Daarmee kan je een DC motor in wijzerzin én in tegenwijzerzin laten draaien, zonder een draad te verleggen.

Met een **transistor** geraakte je niet verder dan aan en uit, want die ene transistor zit tussen de motor en de massa en de stroom kan er dus maar op één manier door. Om te keren zou je de twee draden van de motor moeten omwisselen. Een H-brug doet dat met vier schakelaars.

Waarom die naam?

Teken vier schakelaars in een vierkant met de motor als dwarsbalk in het midden, en je krijgt de vorm van een hoofdletter H. Twee schakelaars hangen aan de plus, twee aan de massa, en de motor zit ertussen.

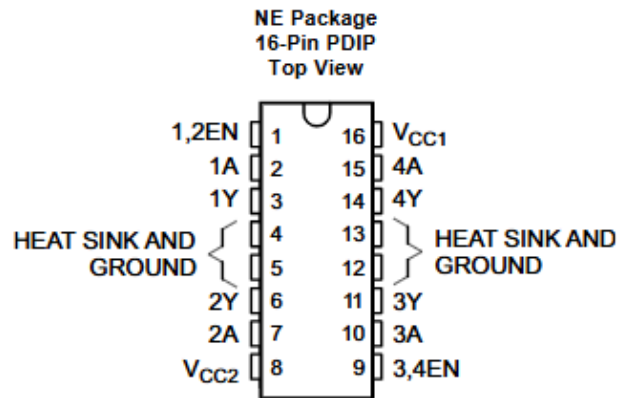
- Sluit je **linksboven** en **rechtsonder**, dan loopt de stroom van links naar rechts door de motor.
- Sluit je **rechtsboven** en **linksonder**, dan loopt hij van rechts naar links, en draait de motor de andere kant op.
- Laat je alles open, dan staat de motor stil.

NOOIT TWEE SCHAKELAARS BOVEN ELKAAR TEGELIJK

Sluit je linksboven en linksonder samen, dan verbind je de plus rechtstreeks met de massa. Dat is kortsluiting, en er loopt dan geen stroom door de motor maar wel dwars door je schakeling. Bij een kant-en-klare H-brug-IC hoef je daar in de praktijk niet aan te denken, omdat je per zijde maar één sturingang hebt en het IC zelf de twee schakelaars aanstuurt.

De L293D

Een H-brug zit kant-en-klaar in een IC, dus je bouwt ze niet uit losse transistoren. Wij gebruiken de **L293D**. Dat is een **dubbele** H-brug, dus je kan er twee DC motoren mee in beide richtingen laten draaien, of vier motoren in dezelfde richting.



Pin Functions

PIN		TYPE	DESCRIPTION
NAME	NO.		
1,2EN	1	I	Enable driver channels 1 and 2 (active high input)
<1:4>A	2, 7, 10, 15	I	Driver inputs, noninverting
<1:4>Y	3, 6, 11, 14	O	Driver outputs
3,4EN	9	I	Enable driver channels 3 and 4 (active high input)
GROUND	4, 5, 12, 13	—	Device ground and heat sink pin. Connect to printed-circuit-board ground plane with multiple solid vias
V _{CC1}	16	—	5-V supply for internal logic translation
V _{CC2}	8	—	Power VCC for drivers 4.5 V to 36 V

🔗 De pinout van de L293D, met de pinfuncties uit de [datasheet](#) .

De twee bruggen zitten verdeeld over de twee zijden van het IC. Per brug heb je drie soorten pinnen:

Soort	Brug 1	Brug 2	Wat het doet
Enable	pin 1 (1,2EN)	pin 9 (3,4EN)	Zet de brug aan. Staat deze laag, dan gebeurt er niets, wat je ook op de ingangen zet.
Ingangen	pin 2 (1A) en pin 7 (2A)	pin 10 (3A) en pin 15 (4A)	Hierop stuur je vanuit de Arduino. Samen bepalen ze de richting.
Uitgangen	pin 3 (1Y) en pin 6 (2Y)	pin 11 (3Y) en pin 14 (4Y)	Hierop hangt de motor.

Daarnaast zijn er twee voedingen en de massa:

- **Vcc1** (pin 16) voedt de logica in het IC en krijgt 5 V.
- **Vcc2** (pin 8) voedt de *motoren*, en dat is de spanning die op je motor terechtkomt. Die mag hoger zijn dan 5 V, want daar hoeft de Arduino niets van te merken.
- **GROUND** zit op pin 4, 5, 12 en 13. Die vier pinnen dienen ook om de warmte af te voeren, dus verbind ze allemaal.

Hoe je de richting kiest

Een uitgang volgt zijn ingang. Zet je 1A hoog en 2A laag, dan staat 1Y hoog en 2Y laag, en loopt de stroom van 1Y door de motor naar 2Y. Draai je die twee om, dan draait de motor om.

1,2EN	1A	2A	Wat de motor doet
HIGH	HIGH	LOW	Draait de ene kant op
HIGH	LOW	HIGH	Draait de andere kant op
HIGH	LOW	LOW	Staat stil, en remt af
HIGH	HIGH	HIGH	Staat stil, en remt af
LOW	maakt niet uit	maakt niet uit	Staat stil, en loopt vrij uit

Brug 1, met de motor tussen 1Y en 2Y

Merk het verschil tussen de laatste twee gevallen op. Zet je beide ingangen gelijk, dan hangen de twee motoraansluitingen op dezelfde spanning en remt de motor af. Zet je de enable laag, dan hangt de motor los en tolt hij uit. Dat is het gedrag dat je wil kunnen kiezen bij een rolruik of een lift.

Snelheid regelen

De ingangen bepalen de richting, en de **enable** bepaalt of er stroom loopt. Zet je op die enable-pin geen vaste HIGH maar een PWM-sigitaal met `analogWrite()`, dan staat de brug snel afwisselend aan en uit en krijgt de motor gemiddeld minder spanning. Zo regel je de snelheid zonder de richting los te laten.

HANG JE ENABLE DUS AAN EEN PIN MET EEN ~

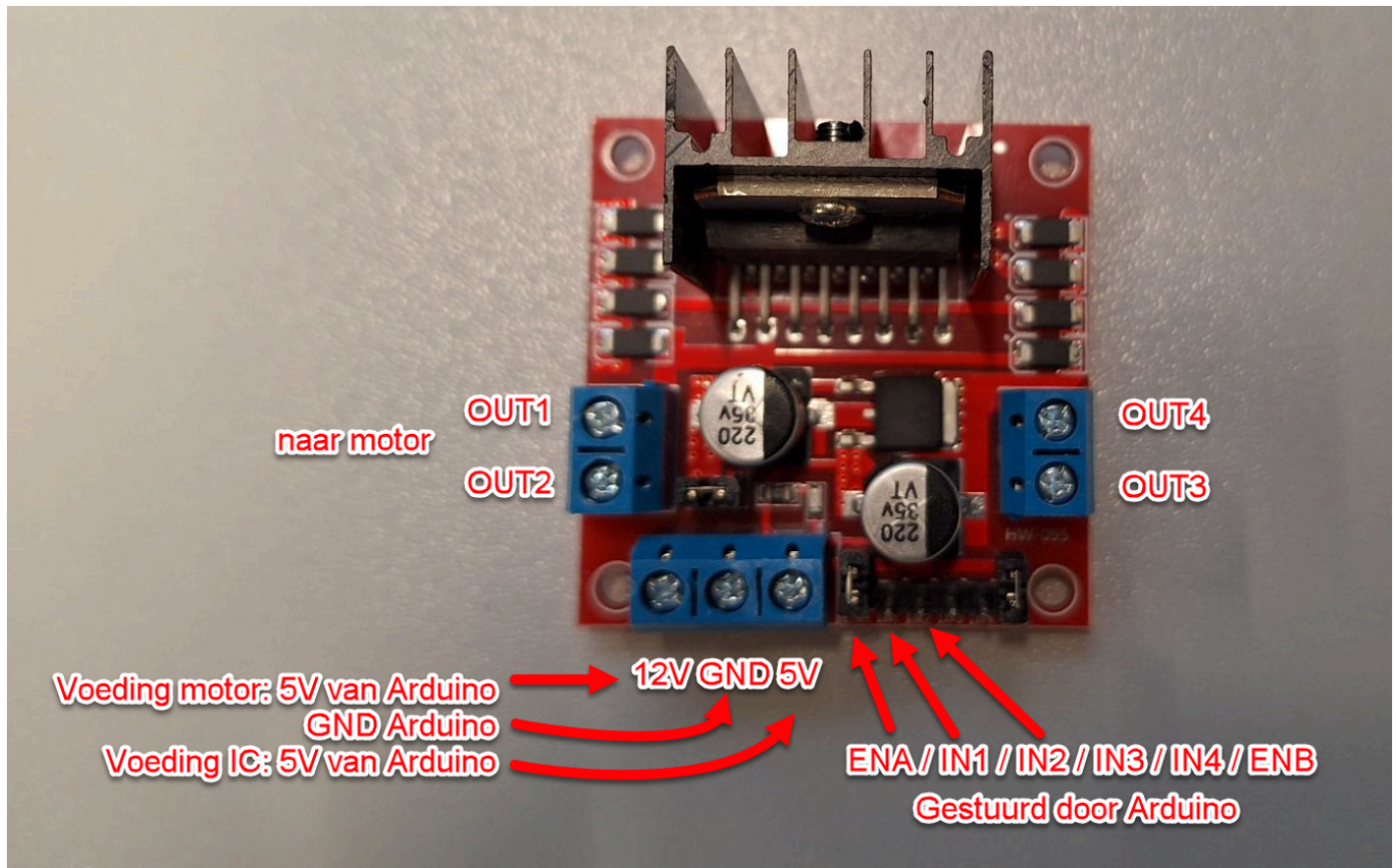
De enable is de pin waar je wil kunnen dimmen, dus die hoort op een PWM-pin: 3, 5, 6, 9, 10 of 11 op de UNO. De ingangen 1A en 2A schakelen alleen maar aan of uit en mogen op eender welke digitale pin.

En de vrijlooptiodes?

Bij de transistor moest je zelf een [vrijlooptiode](#) over de motor zetten. Bij de L293D zitten die diodes al in het IC, en daar staat de **D** in de naam voor. Een kale L293 zonder D heeft ze niet, en dan moet je ze wel zelf voorzien.

De L298N-module

De **L298N** doet hetzelfde als de L293D maar kan veel meer stroom aan. Je koopt hem meestal niet als los IC maar als kant-en-klare module, met de koelplaat, de diodes en de schroefklemmen er al op.



[Klik op de afbeelding om te vergroten](#)

De pinnen heten anders, maar het zijn dezelfde. Deze tabel vertaalt tussen de twee:

L293D	L298N	Functie
1,2EN	ENA	Enable van motor 1
3,4EN	ENB	Enable van motor 2
1A, 2A	IN1, IN2	Sturing van motor 1
3A, 4A	IN3, IN4	Sturing van motor 2
1Y, 2Y	OUT1, OUT2	Uitgangen voor motor 1
3Y, 4Y	OUT3, OUT4	Uitgangen voor motor 2
Vcc1	5V	Voeding voor het IC
Vcc2	12V	Voeding voor de motoren
GROUND	GND	Massa

DE KLEM MET "12V" HOEFT GEEN 12 V TE KRIJGEN

Die opdruk is misleidend. Het is dezelfde Vcc2 als daarnet, de spanning die op je motor terechtkomt. Werkt jouw motor op 5 V, dan zet je daar 5 V op. Zet je een motor van 5 V op 12 V omdat het er zo staat, dan maak je hem stuk.